WEB API

SOICT ONE LOVE. ONE FUTURE.

1

Learning Outcomes

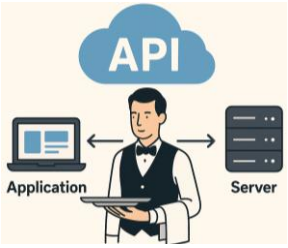
- **Explain** what an API is and how it works in modern web applications.
- **Describe** JSON structure and use it to exchange data.
- **Understand** REST principles
- **Send and handle API requests** using Fetch and Axios.
- **Use Promises and async/await** to manage asynchronous operations.
- **Build simple client–server interactions** (consuming public APIs).
- **Debug and troubleshoot** common API call errors (CORS, network, status codes).

1. Web API Fundamentals
1.1. API
1.2. JSON
1.3. REST

3

1.1. API

- An API (application programming interface) is a set of rules or protocols that enables software applications to communicate with each other.
- An API is like a **restaurant waiter** – you tell the waiter what you want, the waiter communicates with the kitchen, and brings the result back to you.



4

Why Do We Need APIs?

- Modularity
  - breaks large, complex systems into smaller, independent services
  - enables easier maintenance, faster development, and better scalability.
- Security (Access Control):
  - exposes only the necessary endpoints and actions
  - protects sensitive data from unauthorized access
- Interoperability (Integration):
  - provides a communication standard for different technologies and systems
  - allows systems to integrate easily with each other.

Real-World Examples of APIs

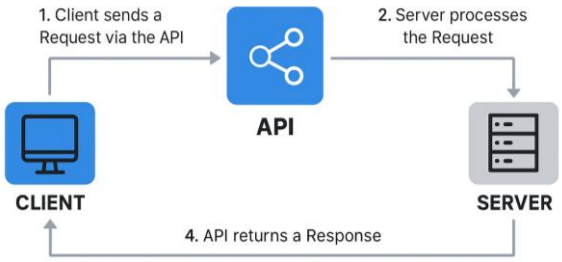
- Daily Apps
  - Google Maps API → show maps inside mobile apps
  - YouTube API → embed video, search results
  - Gmail API → send email
  - Facebook API → login with Facebook
  - Message API → send message
  - Weather API → display current weather
- Finance
  - VNPay API, Momo API, ZaloPay API → payment
  - Stripe API, PayPal API → payment
- E-commerce
  - Tiki Open API → product, order
  - Amazon API → product, order

4 Core Components of a Web API

Component	Meaning	Purpose
URL	Resource address	Identify target
Method	Action type	Define operation
Headers	Metadata	Context, auth, format
Body	Data payload	Send content

How Web APIs Work (Simple Flow)

- APIs act as a communication bridge between a **client** (your app, browser, or system) and a **server** (the backend that stores data or performs actions).
- Flow
  - 1. Client sends a Request via the API
  - 2. API receives the Request
  - 3. Server processes the Request
  - 4. API returns a Response



1.2. JSON

- **JSON (JavaScript Object Notation)** is a lightweight, text-based data format used to store and exchange data between systems.
- Features
  - Human-readable
  - Easy for machines to parse
  - Language-independent
- Structure
  - Objects (key-value pairs)
  - Arrays (ordered lists)
- Note
  - No comments allowed
  - UTF-8 by default

```
{
  "name": "John",
  "age": 25,
  "isStudent": false
}
```

```
{
  "user": {
    "id": 101,
    "name": "Alice"
  },
  "roles": ["admin", "editor"]
}
```


JSON Structure: Object & Array

- Two main structures
  - Objects → key–value pairs
  - Arrays → an ordered list of values
- JSON Object: collection of **key–value pairs** enclosed in { }
  - Format: "key": value
  - Keys are always strings
  - Values can be any JSON data type
  - Key–value pairs are separated by commas
  - Unordered

```
{
  "name": "John",
  "age": 25,
  "isStudent": false
}
```

JSON Problems

- JSON
  - supports: String, Number, Boolean, Null, Object, Array
  - not support: Date, Function, undefined, binary data, regular expression, NaN

- Date → JSON → Date
  - `const obj = {created: new Date() };`
  - `JSON.stringify(obj);`

  `//{ "created": "2025-10-10T05:00:00.000Z" }`
  - `const data = JSON.parse(json);`
  - `data.created = new Date(data.created);`
- `JSON.stringify({ x: undefined }); // "{}"`
- `JSON.stringify({ fn: () => {} });// "{}"`

JSON Nested

- Nested JSON
  - Objects contain objects
  - Objects contain arrays
  - Arrays contain objects

- Example
  - `data.user.profile.city; // "Hanoi"`
  - `data.orders[0].items[1].price; // 20`

```
{
  "user": {
    "id": 101,
    "name": "Alice",
    "profile": {
      "age": 25,
      "city": "Hanoi"
    }
  },
  "orders": [
    {
      "orderId": "A001",
      "items": [
        { "name": "Laptop", "price": 1500 },
        { "name": "Mouse", "price": 20 }
      ]
    }
  ]
}
```

1.3. REST

- REST = Representational State Transfer: is a set of principles used to design Web APIs.
- Restful API: is an API that follows REST principles

Principle	Meaning
Client–Server	UI separate from backend
Stateless	No session on server
Cacheable	Responses can be cached
Uniform Interface	Consistent API design
Layered System	Multiple transparent layers
Code on Demand	Server sends code (optional)

REST: Resource Naming Rules

- Resource: any entity
- Use nouns, not a verb, e.g., users, orders, posts
  - `Good: /users, /orders/123`
  - `Bad: /user, /getUsers, /createOrder`
- Use hierarchical structure:
  - `/users/10/orders`
  - `/orders/5/items`
- Use Path for Resources, Query for Filters
  - `/products → all products`
  - `/products?category=phone → filtered list`
- Use Lowercase & Hyphens: `/user-profiles`
- Avoid Trailing Slashes: `/users` insted of `/users/`

CRUD Mapping

CRUD	HTTP Method	Example	Description
Create	**POST**	/users	Create new user
Read	**GET**	/users/5	Retrieve user
Update	**PUT**	/users/5	Replace entire user
	PATCH	/users/5	Update partitally
Delete	**DELETE**	/users/5	Delete user

Parameters: Identifier (Path) vs Filter (Query)

- Path Parameters = Identifier
  - represent a **specific resource**
  - identify resource by ID/code
  - order is important
  - `GET /users/10`
  - `GET /orders/A001/items/5`
- Query Parameters = Filters/Sorting/Pagination
  - use for **query** (e.g., filtering, sorting, pagination)
  - order doesn't matter
  - `GET /products?category=phone`
  - `GET /posts?sort=desc&page=2&limit=10`
  - ❌ `/products/category/phone (not RESTful)`

HTTP Status Code

Category	Meaning	Examples
1xx	Info	100
2xx	Success	200, 201, 204
3xx	Redirect	301, 302, 304
4xx	Client error	400, 401, 403, 404
5xx	Server error	500, 502, 503, 504

2xx Success Codes (200, 201, 204)

- 200 OK
  - Request succeeded and returns a response body.
  - Return JSON data
  - e.g., GET /users
- 201 Created
  - A new resource was **successfully created**
  - May return newly created object
  - e.g., POST /users (create user)
- 204 No Content
  - Request succeeded **but no response body**
  - e.g., DELETE /users/10

4xx Client Errors (400, 401, 403, 404)

- 400 — Bad Request
  - server **cannot process** the request because of invalid request
  - e.g., POST /users with missing "name" field
- 401 - Unauthorized
  - authentication is **required** but missing or invalid.
  - e.g, GET /users without JWT token
- 403 — Forbidden
  - Client is authenticated but **not allowed** to access
  - e.g., insufficient permissions
- 404 — Not Found
  - requested resource **does not exist**
  - e.g., wrong path, resource not found

5xx Server Errors (500, 503)

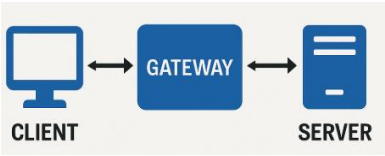
- 500 — Internal Server Error
  - **server itself** encountered an error.
  - e.g., backend bugs, database errors, exception
- 503 — Service Unavailable
  - server is **reachable**, but **temporarily unable** to handle the request.
  - e.g., server overload
- 502 — Bad Gateway (Gateway: Nginx, load balancer)
  - gateway received a bad **response** from server.
  - e.g, server returned invalid response
- 504 — Gateway Timeout
  - gateway **did not receive a response in time** from server
  - e.g., slow backend processing

Gateway (Nginx, Apache, AWS, Azure...)

- Gateway is the smart middle layer that controls how the Client communicates with the Server
  - protect backend server
  - filter request, block attacks
  - direct traffics
  - track errors

```
upstream backend {
    server 127.0.0.1:3000;
    server 127.0.0.1:3001;
}

server {
    listen 80;
    location / { proxy_pass http://backend; }
}
```



21

CORS: When the Browser "Asks Permission"

- CORS is a security mechanism in browsers that controls whether a web page can call APIs from a different origin.
  - Origin = protocol + domain + port
  - e.g., http://app.com and http://api.com are different origins
- Browser sends Preflight request for non-simple requests (e.g., DELETE, PUT, PATCH)
  - OPTIONS /api HTTP/1.1
  - Origin: https://api.com
- Server must reply as follows to allow browser to call API
  - HTTP/1.1 204 No Content
  - Access-Control-Allow-Origin: https://api.com
  - Access-Control-Allow-Methods: GET, POST, PUT
  - Access-Control-Allow-Headers: Content-Type

22

Simple vs Non-Simple Requests

- Simple requests: three conditions
  1. Allowed Methods: GET, POST, HEAD
  2. Allowed Headers (Only simple headers)
     - Accept
     - Accept-Language
     - Content-Language
     - Content-Type (but only 3 specific types below)
  3. Content-Type must be one of:
     - text/plain
     - multipart/form-data
     - application/x-www-form-urlencoded

Browser sends the **request immediately** (no OPTIONS).

2. Fetch API

Introduction

- A modern interface for fetching resources
- Native in all modern browsers
- Advantages
  - simplicity and modernity (promise-based)
  - clear request/response objects for data handling
  - excellent JSON handling
- Disadvantages
  - Error handling complexity (manual status check)
  - Lack of timeout mechanism
  - Manual cookie and credentials handing

Fetch API: Syntax with Async Await

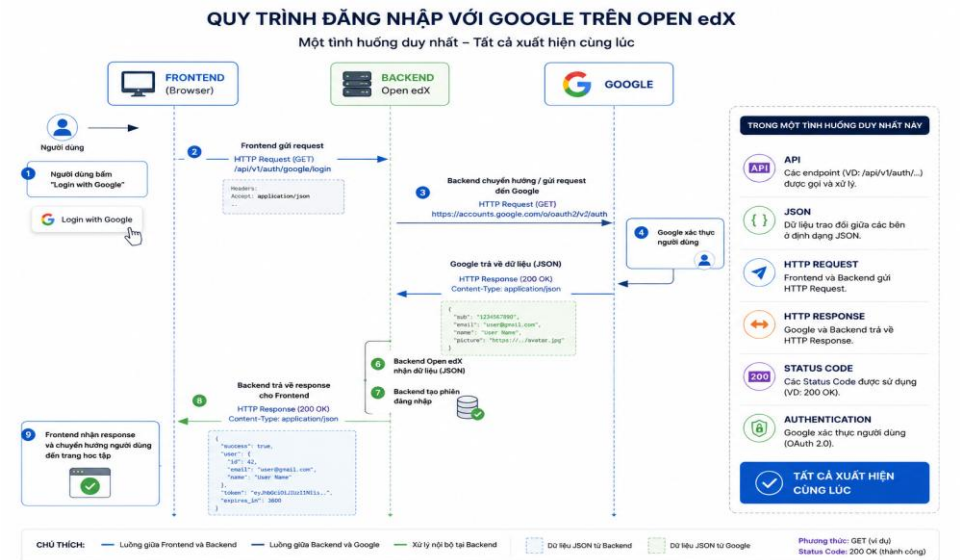
- Writes code like traditional synchronous (blocking) code.
  - async: declares an asynchronous function
  - await: pauses the execution of the async function until the Promise settles
- Two-step process for data access
  - fetch(): return a response object
  - response.json() or response.text to access the data

```
async function fetchData() {
  try {
    const response = await fetch('https://api.example.com/data');
    if (!response.ok) {
      throw new Error(`HTTP error! status: ${response.status}`);
    }
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.error('Fetch error:', error);
  }
}
```

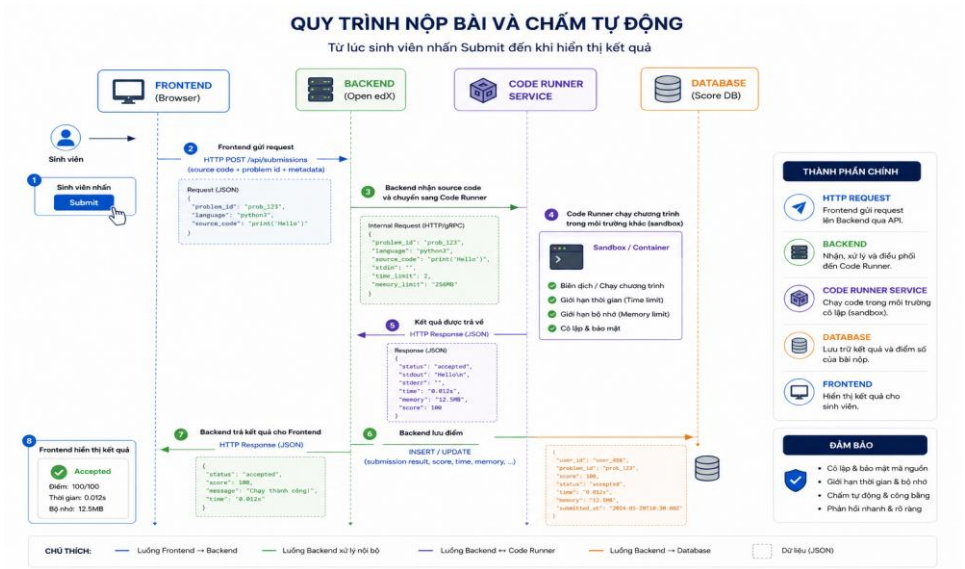
Axios vs Fetch

- Use Axios for: large apps, interceptors, timeouts, clean error handling.
- Use Fetch for: simple requests, modern browsers, lightweight usage without libraries.
- Axios
  - Better error handling (treats non-2xx as errors)
  - Has built-in timeout
  - Cleaner syntax for complex calls
- Fetch
  - Native browser API (no installation)
  - No built-in timeout
  - Requires manual JSON parsing
  - Treats non-2xx responses as success (must check res.ok)

Case Study 1



Case Study 2


QUY TRÌNH NỘP BÀI VÀ CHẤM TỰ ĐỘNG
Từ lúc sinh viên nhấn Submit đến khi hiển thị kết quả
FRONTEND (Browser)
BACKEND (Open edX)
CODE RUNNER SERVICE
DATABASE (Score DB)
Sinh viên
Sinh viên nhấn
Submit
Frontend gửi request
HTTP POST /api/submissions
(source code + problem id + metadata)
Request (JSON)
Backend nhận source code và chuyển sang Code Runner
Internal Request (HTTP/gRPC)
Code Runner chạy chương trình trong môi trường khác (sandbox)
Sandbox / Container
Biên dịch / Chạy chương trình
Giới hạn thời gian (Time limit)
Giới hạn bộ nhớ (Memory limit)
Cô lập & bảo mật
Kết quả được trả về
HTTP Response (JSON)
Response (JSON)
Backend lưu điểm
INSERT / UPDATE
(submission result, score, time, memory, ...)
Backend trả kết quả cho Frontend
HTTP Response (JSON)
Frontend hiển thị kết quả
Accepted
Điểm: 100/100
Thời gian: 0.012s
Bộ nhớ: 12.5MB
THÀNH PHẦN CHÍNH
HTTP REQUEST
Frontend gửi request lên Backend qua API.
BACKEND
Nhận, xử lý và điều phối đến Code Runner.
CODE RUNNER SERVICE
Chạy code trong môi trường cô lập (sandbox).
DATABASE
Lưu trữ kết quả và điểm số của bài nộp.
FRONTEND
Hiển thị kết quả cho sinh viên.
ĐẢM BẢO
Cô lập & bảo mật mã nguồn
Giới hạn thời gian & bộ nhớ
Chấm tự động & công bằng
Phản hồi nhanh & rõ ràng
CHÚ THÍCH:
Luồng Frontend → Backend
Luồng Backend xử lý nội bộ
Luồng Backend ↔ Code Runner
Luồng Backend → Database
Dữ liệu (JSON)